Guía práctica: Carrito de compras con React y Zustand

Oct 1, 2026 · @Juan Pablo

Presentación

Al terminar esta práctica tendrás una tienda en React con un carrito de compras cuyo estado se comparte entre componentes usando Zustand, sin pasar props ni usar Providers.

Resultado de aprendizaje: implementar un estado global en una aplicación React con la librería Zustand, separando datos, lógica y vista.

Duración estimada: 2 a 3 horas.

¿Qué es Zustand?

Zustand es una librería externa y ligera del ecosistema de React para manejar estado global. No viene incluida en React: se instala aparte. Funciona como un tablero central (llamado *store*) donde se guarda la información que comparten varios componentes y las acciones que la modifican. Cualquier componente puede leer ese tablero directamente, sin que otros componentes le pasen la información.

Requisitos previos

- Node.js 18 o superior instalado (verifica con `node -v`).
- Visual Studio Code u otro editor.
- Conocer componentes, props, `useState` y el método `map` de los arreglos.

Qué vas a construir

Una página con tres componentes independientes que comparten el mismo carrito:

Componente	Qué hace	Qué usa del store
`Encabezado`	Muestra cuántos productos hay en el carrito	`items`
`ListaProductos`	Lista los productos y permite agregarlos	`agregarProducto`
`Carrito`	Muestra lo agregado, el total y permite quitar o vaciar	`items`, `eliminarProducto`, `vaciarCarrito`

Paso 1: Crear el proyecto e instalar Zustand

Abre una terminal en la carpeta donde guardas tus proyectos y ejecuta estos comandos uno por uno:

```
# Crea un proyecto React con Vite llamado carrito-zustand
npm create vite@latest carrito-zustand -- --template react
```

ANALISIS Y DESSARROLLO DE SOFTWARE
DESARROLLAR LA SOLUCION DE SOFTWARE

```
# Entra a la carpeta del proyecto
cd carrito-zustand

# Instala las dependencias base de React
npm install

# Instala Zustand como dependencia externa
npm install zustand

# Abre el proyecto en Visual Studio Code
code .
```

Para comprobar que Zustand quedó instalado, abre `package.json` y busca `zustand` dentro de `dependencies`.

Paso 2: Organizar las carpetas

Dentro de `src`, borra `App.css` y crea las carpetas y archivos que faltan hasta tener esta estructura:

```
src/
├── data/
│   └── productos.js ← datos de la tienda
├── store/
│   └── useCarritoStore.js ← estado global (Zustand)
├── components/
│   ├── Encabezado.jsx ← título y contador
│   ├── ListaProductos.jsx ← catálogo
│   └── Carrito.jsx ← carrito y total
├── App.jsx ← une los componentes
├── index.css ← estilos de la app
└── main.jsx ← punto de entrada
```

Cada carpeta tiene una responsabilidad: `data` guarda la información, `store` la lógica del estado y `components` la parte visual.

Paso 3: Crear los datos de la tienda

Este archivo simula la información que más adelante vendría de una base de datos o una

API. **Archivo:** `src/data/productos.js`

```
// Exportamos un arreglo con los productos que se venden en la tienda
export const productos = [
 // Cada producto es un objeto con un id único, un nombre y un precio en pesos
{ id: 1, nombre: 'Camiseta', precio: 45000 },
 { id: 2, nombre: 'Gorra', precio: 30000 },
 { id: 3, nombre: 'Tenis', precio: 180000 },
 { id: 4, nombre: 'Morral', precio: 95000 },
] // Fin del arreglo de productos
```

El `id` es importante: lo usaremos para saber si un producto ya está en el carrito y para eliminarlo.

Paso 4: Crear el store del carrito

Este es el corazón de la práctica. Aquí viven el dato compartido (`items`) y las acciones que lo modifican. Ningún componente guarda su propia copia del carrito.



Archivo: `src/store/useCarritoStore.js`

```
// Importamos la función create, que construye un store de Zustand
import { create } from 'zustand'

// create recibe una función que devuelve el estado inicial y las acciones
// set es la función que Zustand nos da para modificar el estado // El
resultado es un hook personalizado que exportamos para usarlo en los
componentes
export const useCarritoStore = create((set) => ({

 // ===== ESTADO =====
 // items guarda los productos del carrito; al iniciar está vacío
items: [],

 // ===== ACCIONES =====

 // Agrega un producto al carrito; recibe el objeto producto completo
agregarProducto: (producto) =>
 // set recibe el estado actual (state) y devuelve lo que cambia
set((state) => {
 // Buscamos si el producto ya está en el carrito comparando por id
const existe = state.items.find((item) => item.id === producto.id)

 // Si el producto ya existe, solo aumentamos su cantidad
 if (existe) {
 return {
 // Recorremos los items y creamos un arreglo nuevo (nunca modificamos el
original)
 items: state.items.map((item) =>
 item.id === producto.id
 ? { ...item, cantidad: item.cantidad + 1 } // Copia del item con  cantidad + 1
 : item // Los demás items quedan igual
 ),
 }
 }

 // Si no existe, lo agregamos al final con cantidad 1
 return { items: [...state.items, { ...producto, cantidad: 1 }] }  }),

 // Elimina un producto del carrito usando su id
 eliminarProducto: (id) =>
 set((state) => ({
 // filter deja todos los items excepto el que tiene ese id
items: state.items.filter((item) => item.id !== id),
 })),
```

```
 // Vacía el carrito por completo; no necesita el estado anterior
vaciarCarrito: () => set({ items: [] }),

})) // Fin del store
```

Ideas clave para entender este archivo:



- `create` construye el store y devuelve un hook. Por convención, su nombre empieza con `use`.
- `set` reemplaza solo las propiedades que devuelves; el resto del estado se conserva. • Nunca se modifica el arreglo original con `push` o asignaciones directas. Siempre se crea un arreglo nuevo con `map`, `filter` o el operador spread (`...`). Así React detecta el cambio y actualiza la pantalla.

Paso 5: Componente Encabezado

Muestra el título de la tienda y cuántas unidades hay en el carrito. No tiene ninguna relación con la lista de productos, pero se actualiza solo porque lee el mismo store.

Archivo: `src/components/Encabezado.jsx`

```
// Importamos el hook del store para leer el carrito
import { useCarritoStore } from '../store/useCarritoStore'

// Definimos el componente Encabezado
function Encabezado() {
 // Leemos solo items del store usando un selector (state => state.items)
// El componente se vuelve a dibujar solo cuando items cambia
 const items = useCarritoStore((state) => state.items)

 // Sumamos las cantidades de todos los items para obtener el total de unidades
// reduce recorre el arreglo acumulando un valor que empieza en 0  const
totalUnidades = items.reduce((suma, item) => suma + item.cantidad, 0)

 // Devolvemos el JSX del encabezado
 return (
 <header className="encabezado">
 {/* Nombre de la tienda */}
 <h1>Mi Tienda</h1>
 {/* Insignia con el número de unidades en el carrito */}
 <span className="insignia"> {totalUnidades}</span>
 </header>
 )
}

// Exportamos el componente para usarlo en App
export default Encabezado
```

Paso 6: Componente ListaProductos

Muestra el catálogo. No guarda ningún dato: solo toma la acción `agregarProducto` del store y

la ejecuta al hacer clic.

Archivo: `src/components/ListaProductos.jsx`

```
// Importamos el arreglo de productos de la tienda
import { productos } from '../data/productos'
// Importamos el hook del store
import { useCarritoStore } from '../store/useCarritoStore'
```





```
// Definimos el componente ListaProductos
function ListaProductos() {
 // Del store solo necesitamos la acción para agregar
 const agregarProducto = useCarritoStore((state) => state.agregarProducto)

 return (
 // section agrupa todo el catálogo; panel le da apariencia de tarjeta
<section className="panel">
 <h2>Productos</h2>

 {/* Recorremos los productos y creamos una fila por cada uno */}
{productos.map((producto) => (
 // key ayuda a React a identificar cada elemento de la lista  <div
key={producto.id} className="fila">
 {/* Bloque con el nombre y el precio */}
 <div>
 <p className="fila-nombre">{producto.nombre}</p>
 {/* toLocaleString('es-CO') da formato de pesos: 45.000 */}  <p
className="fila-detalle">${producto.precio.toLocaleString('es CO')}</p>
 </div>

 {/* Al hacer clic, enviamos el producto completo a la acción del store */}  <button
className="btn btn-primario" onClick={() =>
agregarProducto(producto)}>
 Agregar
 </button>
 </div>
 ))}
 </section>
 )
}

// Exportamos el componente
export default ListaProductos
```

Ojo: en `onClick` se escribe `() => agregarProducto(producto)` y no `agregarProducto(producto)`. Sin la función flecha, la acción se ejecutaría al dibujar la página y no al hacer clic.

Paso 7: Componente Carrito

Muestra los productos agregados, calcula el total y permite quitar productos o vaciar el

carrito. **Archivo:** `src/components/Carrito.jsx`

```
// Importamos el hook del store
import { useCarritoStore } from '../store/useCarritoStore'

// Definimos el componente Carrito
function Carrito() {
 // Leemos el dato items y las dos acciones que vamos a usar
 const items = useCarritoStore((state) => state.items)
 const eliminarProducto = useCarritoStore((state) => state.eliminarProducto)
const vaciarCarrito = useCarritoStore((state) => state.vaciarCarrito)
```



```
 // Calculamos el total: precio por cantidad de cada item, sumado
const total = items.reduce(
 (suma, item) => suma + item.precio * item.cantidad, // Suma parcial de
cada item
 0 // Valor inicial de la suma
 )

 // Si el carrito no tiene items, mostramos un mensaje y terminamos aquí
if (items.length === 0) {
 return (
 <section className="panel">
 <h2>Carrito</h2>
 <p className="vacio">Tu carrito está vacío</p>
 </section>
 )
 }

 // Si hay items, mostramos la lista, el total y el botón de vaciar
return (
 <section className="panel">
 <h2>Carrito</h2>

 {/* Una fila por cada item del carrito */}
 {items.map((item) => (
 <div key={item.id} className="fila">
 <div>
 <p className="fila-nombre">{item.nombre}</p>
 {/* Mostramos la cantidad y el subtotal de ese producto */}  <p
className="fila-detalle">
 {item.cantidad} x ${item.precio.toLocaleString('es-CO')}  </p>
 </div>

 {/* Enviamos el id del item a la acción eliminarProducto */}  <button
className="btn btn-secundario" onClick={() =>
eliminarProducto(item.id)}>
 Quitar
 </button>
 </div>
 ))}

 {/* Bloque con el total a pagar */}
 <div className="total">
```

```
 <span>Total</span>
 <strong>${total.toLocaleString('es-CO')}</strong>
 </div>

 {/* Vacía todo el carrito; no necesita parámetros */}
 <button className="btn btn-secundario btn-bloque" onClick={vaciarCarrito}>
Vaciar carrito
 </button>
 </section>
 )
}
```



```
// Exportamos el componente
export default Carrito
```

Aquí `onClick={vaciarCarrito}` no lleva función flecha porque la acción no recibe parámetros: se pasa la referencia y React la ejecuta al hacer clic.

Paso 8: Unir todo en App y revisar main

`App` solo organiza la página. Fíjate que no pasa ninguna prop: cada componente se conecta al store por su cuenta.

Archivo: `src/App.jsx` (reemplaza todo su contenido)

```
// Importamos los tres componentes de la tienda
import Encabezado from './components/Encabezado'
import ListaProductos from './components/ListaProductos'
import Carrito from './components/Carrito'

// Componente principal de la aplicación
function App() {
 return (
 // Contenedor general que centra la página
 <div className="app">
 {/* Encabezado con el contador del carrito */}
 <Encabezado />

 {/* Zona principal con dos columnas: catálogo y carrito */}
<main className="contenido">
 <ListaProductos />
 <Carrito />
 </main>
 </div>
 )
}

// Exportamos App para que main.jsx la muestre
export default App
```

Archivo: `src/main.jsx` (normalmente ya viene así; solo verifícalo)

```
// StrictMode ayuda a detectar errores durante el desarrollo
```

```
import { StrictMode } from 'react'
// createRoot conecta React con el HTML de la página
import { createRoot } from 'react-dom/client'
// Importamos los estilos globales
import './index.css'
// Importamos el componente principal
import App from './App.jsx'

// Buscamos el div con id root en index.html y dibujamos App dentro
createRoot(document.getElementById('root')).render(
 <StrictMode>
 <App />
 </StrictMode>,
)
```


Observa que `App` no está envuelta en ningún Provider. Con Context API tendrías que agregar uno aquí; con Zustand no hace falta.

Paso 9: Agregar los estilos Opcional

Borra todo el contenido que trae `src/index.css` y reemplázalo por este. Es un estilo minimalista: fondo claro, tarjetas blancas, un solo color de acento y mucho espacio en blanco. Los colores están en variables para que puedas cambiar el tema en un solo lugar.

Archivo: `src/index.css`

```
/* ===== VARIABLES DE COLOR Y MEDIDAS ===== */
:root {
 --fondo: #f5f5f4; /* Gris muy claro para el fondo de la página */
--tarjeta: #ffffff; /* Blanco para los paneles */
 --texto: #1c1917; /* Casi negro para el texto principal */
--texto-suave: #78716c; /* Gris para textos secundarios */
 --borde: #e7e5e4; /* Gris claro para bordes y separadores */
--acento: #0f766e; /* Verde azulado como color principal */
--acento-hover: #115e59; /* Tono más oscuro al pasar el mouse */
--radio: 12px; /* Redondeo de las esquinas */
}

/* ===== REINICIO BÁSICO ===== */
* {
 margin: 0; /* Quita márgenes por defecto del navegador */  padding: 0; /*
Quita rellenos por defecto */
 box-sizing: border-box; /* El ancho incluye padding y borde */ }

/* ===== CUERPO DE LA PÁGINA ===== */
body {
 font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif; /* Fuente
del sistema */
 background-color: var(--fondo); /* Fondo gris claro */
 color: var(--texto); /* Color de texto principal */  line-height: 1.5;
/* Espaciado cómodo entre líneas */ }

/* ===== CONTENEDOR GENERAL ===== */
```

```
.app {
 max-width: 960px; /* Ancho máximo para pantallas grandes */  margin:
0 auto; /* Centra el contenido horizontalmente */  padding: 2rem 1rem;
/* Espacio alrededor del contenido */ }

/* ===== ENCABEZADO ===== */
.encabezado {
 display: flex; /* Pone título e insignia en una fila */  justify-content:
space-between; /* Título a la izquierda, insignia a la derecha  */
 align-items: center; /* Centra verticalmente */
 margin-bottom: 2rem; /* Separa el encabezado del contenido */ }
```



```
.encabezado h1 {
 font-size: 1.75rem; /* Tamaño del título */
 font-weight: 600; /* Grosor semi negrita */
 letter-spacing: -0.02em; /* Letras ligeramente más juntas */
}

/* Insignia del contador del carrito */
.insignia {
 background-color: var(--tarjeta); /* Fondo blanco */
 border: 1px solid var(--borde); /* Borde suave */
 border-radius: 999px; /* Forma de píldora */
 padding: 0.4rem 1rem; /* Espacio interno */
 font-weight: 500; /* Texto un poco más grueso */ }

/* ===== DISTRIBUCIÓN EN COLUMNAS ===== */
.contenido {
 display: grid; /* Usamos CSS Grid */  grid-template-columns: repeat(auto-fit,
minmax(280px, 1fr)); /* 2 columnas o 1 en  móvil */
 gap: 1.5rem; /* Espacio entre paneles */  align-items: start; /* Cada panel con su
propia  altura */
}

/* ===== PANELES (TARJETAS) ===== */
.panel {
 background-color: var(--tarjeta); /* Fondo blanco */
 border: 1px solid var(--borde); /* Borde suave */
 border-radius: var(--radio); /* Esquinas redondeadas */
 padding: 1.5rem; /* Espacio interno */
}

.panel h2 {
 font-size: 1.1rem; /* Título del panel discreto */
 font-weight: 600; /* Semi negrita */
 margin-bottom: 1rem; /* Separación con la lista */
}

/* ===== FILAS DE PRODUCTOS ===== */
.fila {
 display: flex; /* Texto y botón en una fila */  justify-content: space-between; /*
Texto a la izquierda, botón a la derecha */  align-items: center; /* Centra
verticalmente */
```

```
 padding: 0.75rem 0; /* Espacio arriba y abajo */  border-bottom:
1px solid var(--borde); /* Línea separadora */
}

.fila:last-of-type {
 border-bottom: none; /* La última fila no lleva línea */
}

.fila-nombre {
```



```
font-weight: 500; /* Nombre del producto destacado */ }

.fila-detalle {
 font-size: 0.875rem; /* Texto más pequeño */
 color: var(--texto-suave); /* Color gris */
}

/* ===== BOTONES ===== */
.btn {
 font: inherit; /* Usa la misma fuente de la página */  font-size:
0.875rem; /* Tamaño de texto del botón */
 font-weight: 500; /* Texto semi grueso */
 padding: 0.5rem 1rem; /* Espacio interno */
 border-radius: 8px; /* Esquinas redondeadas */
 cursor: pointer; /* Manito al pasar el mouse */
 transition: background-color 0.2s, border-color 0.2s; /* Cambio de color suave */
}

/* Botón principal: relleno con el color de acento */
.btn-primario {
 background-color: var(--acento); /* Fondo verde azulado */
 color: #ffffff; /* Texto blanco */
 border: 1px solid var(--acento); /* Borde del mismo color */ }

.btn-primario:hover {
 background-color: var(--acento-hover); /* Más oscuro al pasar el mouse */
}

/* Botón secundario: solo borde, fondo transparente */
.btn-secundario {
 background-color: transparent; /* Sin relleno */
 color: var(--texto); /* Texto oscuro */
 border: 1px solid var(--borde); /* Borde gris */
}

.btn-secundario:hover {
 border-color: var(--texto-suave); /* Borde más visible al pasar el mouse */ }

/* Botón que ocupa todo el ancho del panel */
.btn-bloque {
 width: 100%; /* Ancho completo */
 margin-top: 1rem; /* Separación con el total */
}
```

```
/* ===== TOTAL DEL CARRITO ===== */
.total {
 display: flex; /* Etiqueta y valor en una fila */  justify-content:
space-between; /* Extremos opuestos */
 margin-top: 1rem; /* Separación con la lista */  padding-top: 1rem; /*
Espacio interno superior */  border-top: 2px solid var(--texto); /* Línea
fuerte que marca el total */
```



```
font-size: 1.1rem; /* Texto un poco más grande */ }

/* ===== MENSAJE DE CARRITO VACÍO ===== */
.vacio {
 color: var(--texto-suave); /* Texto gris */
 text-align: center; /* Centrado */
 padding: 2rem 0; /* Espacio arriba y abajo */
}
```

Para cambiar el color principal de toda la tienda, basta con modificar `--acento` y `--acento-hover` en `:root`.

Paso 10: Ejecutar y comprobar

En la terminal, dentro de la carpeta del proyecto:

```
# Inicia el servidor de desarrollo de Vite
npm run dev
```

Abre en el navegador la dirección que muestra la terminal (normalmente `http://localhost:5173`) y marca cada prueba a medida que la compruebes:

- ☐ Al cargar la página, el carrito dice "Tu carrito está vacío" y la insignia muestra 0.
- ☐ Al agregar una Camiseta, aparece en el carrito y la insignia sube a 1.
- ☐ Al agregar otra Camiseta, no se duplica la fila: la cantidad pasa a 2.
- ☐ El total se recalcula con cada producto agregado.
- ☐ El botón Quitar elimina solo ese producto.
- ☐ El botón Vaciar carrito deja la insignia en 0 y muestra el mensaje de vacío. • ☐ En una pantalla angosta (o con la ventana reducida), los dos paneles se ubican uno debajo del otro.

Errores frecuentes

Síntoma	Causa probable	Solución
`Failed to resolve import "zustand"`	Zustand no se instaló	Ejecuta `npm install zustand`
`Failed to resolve import "./App.css"`	Se borró App.css pero quedó la importación	Reemplaza App.jsx por el código del paso 8
Pantalla en blanco	Error de escritura en un componente	Abre la consola del navegador (F12) y lee el

		error
Los productos se agregan al cargar la página	Falta la función flecha en `onClick`	Usa `onClick={() => agregarProducto(producto)}`
La cantidad no aumenta, se duplica la fila	La comparación por `id` está mal escrita	Revisa `item.id === producto.id` en el store

Proyectó: Instructor Juan Pablo Chamizzo Vega – S

ANALISIS Y DESSARROLLO DE SOFTWARE
DESARROLLAR LA SOLUCION DE SOFTWARE

ACTIVIDAD DE APRENDIZAJE

Preguntas de reflexión Inicial

Responde o en el README del proyecto:

1. ¿Por qué `Encabezado` se actualiza cuando agregas un producto, si no recibe ninguna prop?
2. ¿Qué tendrías que hacer para lograr lo mismo usando solo `useState` en `App`? 3. ¿Para qué sirve el selector `(state) => state.items` al usar el hook?
4. ¿Por qué en el store se crean arreglos nuevos con `map` y `filter` en lugar de modificar `items` directamente?
5. ¿Zustand es parte de React? Explica la diferencia entre una librería propia de React y una del ecosistema.

Retos de extensión

Cada reto se resuelve agregando una acción en el store y usándola en un componente:

1. **Disminuir cantidad:** agrega un botón "**−**" que reste una unidad. Si la cantidad llega a 0, el producto debe salir del carrito.

2. **Subtotal por producto:** muestra en cada fila del carrito el precio multiplicado por la cantidad. 3. **Carrito persistente:** usa el middleware `persist` de Zustand para que el carrito no se pierda al recargar la página. Pista: `import { persist } from 'zustand/middleware'`. 4. **Productos desde una API:** reemplaza `productos.js` por una petición con `fetch` a `https://fakestoreapi.com/products` y adapta los nombres de las propiedades.

Entregable

Sube tu evidencia a un repositorio de GitHub con un README que incluya una captura de la tienda funcionando y las respuestas a las preguntas de reflexión.

Proyectó: Instructor Juan Pablo Chamizzo Vega – SENA 2026